

المدرسة الوطنية للعلوم التطبيقية بتطوان
ⵜⵉⵍⵍⵉⵏ ⵜⴰⵏⴰⵎⵓⵔⵜ ⵏ ⵜⵙⵙⵏⴰⵢⵉⵏ ⵜⵉⵎⵙⵙⵓⵔⵉⵏ ⴰⵜⵟⵡⴰⵏ
Ecole Nationale des Sciences Appliquées de Tétouan

École Nationale des Sciences Appliquées de Tétouan
Département d'Intelligence Artificielle et de Digitalisation IAD
Science des Données, Big Data et IA

ORACLE

Système de monitoring et d'analyse décisionnelle pour le secteur bancaire

Transformation des données bancaires en indicateurs décisionnels.

Préparé par:
Aymane Hayoun
Zakaria Tahiri
Yassir Adila

Supervisé par:
Pr. Anas Outair

Filière : BDIA2

Année académique: 2025–2026

Contents

Remerciements **3**

1 Introduction **4**
1.1 Context 4
1.2 Problématique 4
1.3 Objectifs du projet 5

2 Source de données principale **6**
2.1 Présentation de la source de données 6
2.2 Structure et contenu des données 6
2.3 Adaptation pour le projet 7

3 Architecture et Stack Technique **8**
3.1 Architecture du système 8
3.2 Architecture Medallion 9
3.3 Flux de traitement des données 10
3.4 Stack technique 10

4 Architecture et Flux de Données **12**
4.1 Architecture des Tables 12
4.1.1 Zone de Staging – Couche Bronze 12
4.1.2 Zone de Qualité – Couche Silver 12
4.1.3 Entrepôt de Données – Schéma en Étoile (Couche Gold) 13
4.1.4 Table de Surveillance 14
4.2 Packages PL/SQL 14
4.2.1 Package Technique : `PKG_DATA_CLEANING` 14
4.2.2 Package Métier : `PKG_BUSINESS_LOGIC` 14
4.3 Mécanisme de Surveillance : Trigger 15
4.3.1 `TRG_DETECT_CRITICAL_RISK` 15
4.4 Flux de Données Global 16

5 Visualisation, Modélisation et Automatisation **17**
5.1 Connexion Oracle – Power BI 17
5.2 Modélisation des données 18
5.3 Dashboard de monitoring bancaire 19
5.3.1 Indicateurs clés (KPI) 19
5.3.2 Analyses visuelles 19
5.3.3 Analyses complémentaires 20

5.3.4 Tableau de monitoring opérationnel 20
5.3.5 Interactivité et filtres 20
5.4 Automatisation et actualisation des rapports 21

6 Conclusion **24**

7 Références **25**

Remerciements

Nous tenons à exprimer notre sincère gratitude à notre professeur, Pr. Anas Outair, pour son enseignement dévoué et la structure complète du cours de Bases de données avancées. Ses explications sur les concepts fondamentaux des bases de données et les implémentations pratiques en SQL et PL/SQL sur Oracle ont été essentielles à notre compréhension.

Les solides bases acquises durant ce cours nous ont permis de reprendre ces principes et de les appliquer directement dans ce projet. Ce travail sur la mise en place d'un système de monitoring et d'analyse bancaire est une application concrète des techniques et méthodologies qu'il nous a enseignées tout au long du semestre, et nous lui sommes reconnaissants pour les outils académiques fournis qui nous ont aidés à réussir ce projet.

Année académique 2025–2026

Chapter 1

Introduction

1.1 Context

Dans le secteur bancaire, la quantité de données générées quotidiennement est massive et diversifiée : transactions financières, informations sur les clients, mouvements de comptes, prêts, et autres indicateurs financiers. Une analyse efficace de ces données est indispensable pour assurer la stabilité de la banque, optimiser les décisions stratégiques et détecter rapidement des comportements anormaux ou suspects.

Pour répondre à ce besoin, nous avons choisi de concevoir un système de monitoring et d'analyse bancaire, capable de fournir une vision globale de l'activité de la banque. Contrairement à un projet orienté développement web ou applicatif, qui se concentrerait sur l'interface utilisateur ou les services en ligne, notre choix s'est orienté vers une approche analytique, en cohérence avec notre formation en Data Science et Intelligence Artificielle. L'objectif est de travailler directement sur les données brutes pour les transformer en informations décisionnelles fiables.

Nous avons donc opté pour Oracle Database comme Data Warehouse, afin de centraliser les données, appliquer des transformations et préparer les informations pour la visualisation, plutôt que de simplement créer une base de données transactionnelle classique. Ce choix permet de simuler un environnement réel où un système OLTP alimente un système OLAP, tout en respectant les bonnes pratiques de gestion et d'analyse de données.

1.2 Problématique

Dans un environnement bancaire réel, les données sont souvent dispersées, hétérogènes et non standardisées, ce qui complique leur exploitation. L'absence d'un système centralisé peut entraîner :

- des erreurs et incohérences dans l'analyse des transactions,
- une visibilité limitée sur les indicateurs clés de performance,
- des difficultés à détecter rapidement les anomalies ou comportements suspects.

Pour répondre à cette problématique, notre projet a mis en place une solution complète :

- Centralisation des données : les données issues d'un système OLTP simulé (PostgreSQL) sont transférées vers un Data Warehouse Oracle.
- Transformation et standardisation : à l'aide de scripts SQL et PL/SQL, les données sont nettoyées, enrichies et préparées pour l'analyse.
- Visualisation et décision : un tableau de bord interactif permet de suivre les indicateurs clés et de détecter les anomalies en temps réel.

Ainsi, nous proposons un système capable de transformer des données brutes en un outil décisionnel fiable et structuré.

1.3 Objectifs du projet

L'objectif principal de ce projet est de concevoir un système analytique complet permettant de suivre et d'analyser l'activité bancaire. Les objectifs détaillés sont :

- Mettre en place une architecture Medallion (Bronze, Silver, Gold) dans un Data Warehouse Oracle pour gérer le flux des données depuis leur réception jusqu'à leur préparation pour l'analyse.
- Appliquer les règles métier et transformations sur les données à l'aide de SQL et PL/SQL, afin de garantir leur qualité et leur cohérence.
- Produire un tableau de bord interactif offrant une vision globale de la banque : volumes de transactions, profils clients, indicateurs de risque et alertes sur comportements anormaux.
- Garantir que le système soit évolutif et prêt pour l'automatisation, permettant une mise à jour régulière des indicateurs et des rapports.

Ce projet illustre ainsi la transformation des données brutes en un outil décisionnel dynamique, aligné avec les pratiques réelles en banque et les concepts étudiés dans notre formation.

Chapter 2

Source de données principale

2.1 Présentation de la source de données

La principale source de données utilisée dans ce projet est un fichier CSV simulant les informations bancaires clients et leurs transactions. Ce fichier contient des données détaillées sur les clients, les comptes bancaires, les transactions, les prêts et les cartes de crédit, ainsi que des informations liées aux retours clients et aux anomalies détectées.

Bien que nous n'ayons pas collecté ces données directement auprès d'une banque réelle, ce dataset est représentatif des données que l'on peut retrouver dans un système bancaire opérationnel. Il nous a permis de simuler un environnement réaliste pour l'alimentation de notre système OLTP et, par extension, du Data Warehouse Oracle.

2.2 Structure et contenu des données

Le fichier CSV contient plusieurs types d'informations, regroupés en catégories principales:

- ***Informations clients*** : identifiant, nom, prénom, âge, genre, adresse, ville, contacts, email.
- ***Informations de compte bancaire*** : type de compte, solde, date d'ouverture, dernière transaction.
- ***Transactions*** : identifiant, date, type, montant, solde après transaction, agence associée.
- ***Prêts*** : identifiant, montant, type, taux d'intérêt, durée, date de décision, statut.
- ***Cartes bancaires*** : identifiant, type, limite de crédit, solde, paiement minimum, historique de paiement.
- ***Feedbacks et anomalies*** : identifiant du feedback, type, statut de résolution, dates, indicateur d'anomalie.

Chaque enregistrement du fichier représente un client et ses interactions avec la banque

sur plusieurs services, ce qui permet de créer des analyses complètes et d'identifier des comportements inhabituels ou à risque.

2.3 Adaptation pour le projet

Pour les besoins du projet, ce fichier CSV a été utilisé comme source OLTP simulée. Les données ont été importées dans PostgreSQL afin de reproduire un environnement opérationnel, avant d'être transférées et transformées dans le Data Warehouse Oracle.

Cette démarche nous a permis de :

- Appliquer les connaissances en SQL et PL/SQL pour nettoyer et transformer les données.
- Simuler un flux réel de données, du système opérationnel (OLTP) vers le système analytique (OLAP).
- Créer des indicateurs décisionnels fiables pour le suivi de l'activité bancaire et la détection d'anomalies.

Chapter 3

Architecture et Stack Technique

3.1 Architecture du système

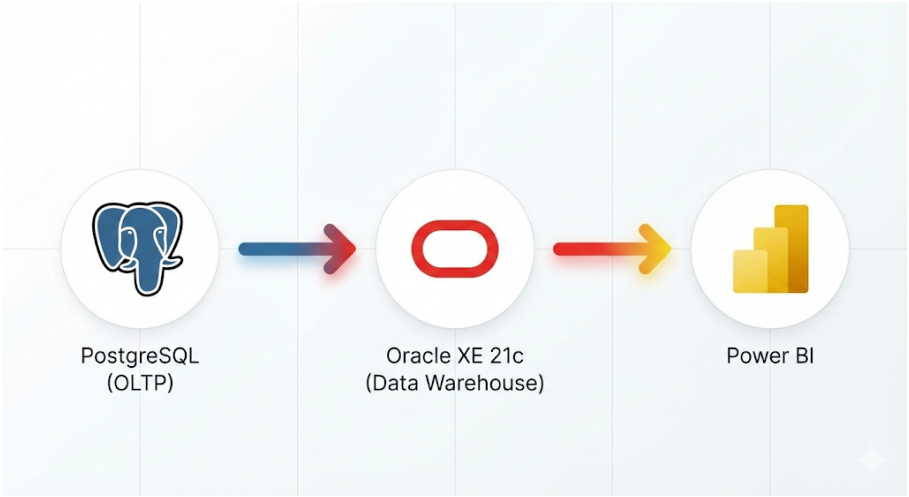


Figure 3.1: Architecture du système

Pour répondre aux besoins d'analyse et de supervision des activités bancaires, nous avons conçu une architecture analytique complète, reposant sur une séparation claire entre le système opérationnel et le système analytique. Cette distinction est essentielle afin de garantir que le traitement analytique n'impacte pas le système source et que les données utilisées pour la prise de décision soient fiables et cohérentes.

- ***Système OLTP (Online Transaction Processing) – PostgreSQL*** :
  Il s'agit de notre système opérationnel simulé. Il contient l'ensemble des données brutes provenant du fichier CSV, incluant les informations clients, comptes, transactions, prêts, cartes bancaires, et anomalies. Cette couche représente le fonctionnement classique d'une banque, où les transactions sont enregistrées au fur et à mesure qu'elles se produisent.

- ***Data Warehouse Oracle (OLAP – Online Analytical Processing)*** :
  Cette couche analytique centralise et organise les données en vue d'analyses com-

plexes. L'ensemble des transformations, calculs et indicateurs décisionnels sont réalisés dans ce système, permettant d'extraire de la valeur à partir des données brutes sans perturber le système opérationnel.

Cette architecture favorise la sécurité, la scalabilité et la performance des traitements, tout en respectant les bonnes pratiques de gestion et d'analyse de données.

Pour transférer les données depuis le système opérationnel PostgreSQL vers le Data Warehouse Oracle, nous avons choisi une approche simple et robuste reposant sur un fichier CSV intermédiaire. Les étapes sont les suivantes :

Extraction depuis PostgreSQL : les données brutes sont exportées sous forme de fichier CSV. Ce fichier contient l'ensemble des enregistrements à transférer, éventuellement horodatés pour permettre une gestion incrémentale du flux.

Chargement dans Oracle (Bronze Layer) : le fichier CSV est ensuite chargé dans la table de staging BRZ_BANKING_IMPORT à l'aide de SQL*Loader. Cette table constitue la couche Bronze, recevant les données brutes sans transformation, conformément à l'architecture Medallion.

Traçabilité et réutilisation : chaque transfert est documenté avec le fichier source et l'horodatage d'ingestion, ce qui permet de rejouer le flux ou d'ajouter de nouvelles données de manière incrémentale.

Cette méthode garantit un flux clair et contrôlé des données entre PostgreSQL et Oracle, tout en respectant les bonnes pratiques de gestion des données dans un Data Warehouse décisionnel.

3.2 Architecture Medallion

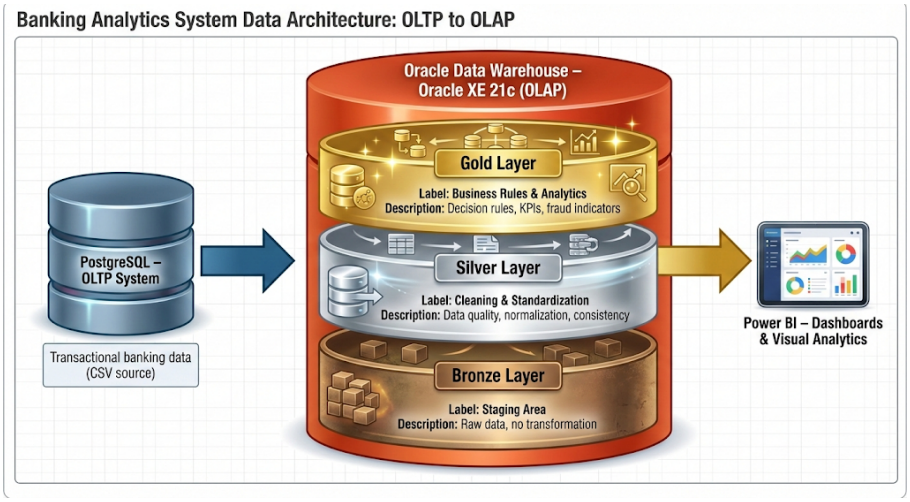


Figure 3.2: Architecture Medallion

Au sein du Data Warehouse Oracle, nous avons mis en place une architecture Medallion en trois couches, permettant un traitement structuré et progressif des données :

1. Bronze Layer – Zone de réception des données :

Cette couche reçoit les données telles qu'elles sont extraites du système OLTP, sans aucune transformation.

L'objectif est de conserver un historique complet et fidèle des données brutes, afin de pouvoir revenir à la source si nécessaire.

2. Silver Layer – Nettoyage et standardisation :

Les données sont filtrées pour éliminer les valeurs manquantes ou incohérentes.

Les formats et unités sont standardisés pour assurer la cohérence entre les différentes catégories de données.

Cette étape garantit que seules des données de qualité sont utilisées pour les analyses décisionnelles.

3. Gold Layer – Couche décisionnelle :

Les règles métier sont appliquées à cette étape pour calculer les indicateurs clés et détecter les anomalies.

Les données sont préparées et structurées de manière optimale pour la visualisation dans Power BI et pour la prise de décision stratégique.

Cette organisation permet de représenter un flux clair et transparent, depuis la collecte des données brutes jusqu'à leur exploitation dans un contexte décisionnel.

3.3 Flux de traitement des données

Le traitement des données suit un flux structuré et progressif:

- **Bronze / Staging** : réception et stockage des données brutes.
- **Silver / Cleaning** : nettoyage, standardisation et enrichissement des données.
- **Gold / Business Rules** : application des règles métier et préparation pour l'analyse décisionnelle.

Ce flux permet de transformer les données brutes en informations fiables et exploitables, tout en offrant un contrôle sur chaque étape de traitement.

3.4 Stack technique

Pour implémenter cette architecture, nous avons utilisé un stack technique complet et adapté aux besoins analytiques :

PostgreSQL : gestion du système OLTP simulé, permettant de stocker et de manipuler les données brutes.

Oracle XE 21c : Data Warehouse OLAP pour la centralisation, la transformation et

l'analyse des données.

Oracle SQL Developer : développement et exécution des scripts SQL et PL/SQL pour le nettoyage, les transformations et l'application des règles métier.

Power BI : visualisation interactive des données et création de tableaux de bord décisionnels dynamiques.

Ce stack a permis de mettre en pratique les concepts étudiés dans le module de Bases de données avancées, notamment la structuration, le nettoyage et la transformation des données, tout en assurant la création d'un système décisionnel robuste et évolutif.

Chapter 4

Architecture et Flux de Données

L'architecture proposée repose sur une approche multi-couches inspirée du paradigme de l'architecture médaillon (Bronze-Silver-Gold). Cette méthodologie permet de structurer le processus de transformation des données selon trois niveaux : l'ingestion brute, la qualité et la standardisation, puis l'enrichissement analytique. L'objectif principal consiste à transformer des données bancaires brutes provenant de la base PostgreSQL en informations décisionnelles exploitables, tout en garantissant la traçabilité, la qualité et la sécurité à chaque étape.

4.1 Architecture des Tables

4.1.1 Zone de Staging – Couche Bronze

La table `BRZ_BANKING_IMPORT` constitue la zone d'atterrissage initiale des données brutes extraites directement de la base PostgreSQL source. Cette couche se caractérise par :

- **Format permissif** : Tous les champs sont définis en `VARCHAR2(4000)`, permettant le chargement des données PostgreSQL sans erreur, même en présence de formats incohérents (dates mal formatées, valeurs numériques contenant du texte, champs vides). Cette approche évite que l'ensemble du processus échoue à cause d'une seule ligne corrompue.
- **Traçabilité** : Conservation d'une copie fidèle des données sources avant toute modification. Cette caractéristique permet de réaliser des audits, facilite le débogage et offre la possibilité de retraiter l'ensemble du pipeline si de nouvelles règles métier sont définies.
- **Horodatage** : Chaque enregistrement est accompagné d'un timestamp d'insertion, permettant de tracer précisément quand chaque lot de données a été extrait de PostgreSQL et chargé.

4.1.2 Zone de Qualité – Couche Silver

La table `SLV_BANKING_DATA` représente la couche où les données subissent leur première transformation significative. Elle contient les données nettoyées, typées correctement,

validées et dédoublonnées :

- **Typage sémantique** : Application de types appropriés (`NUMBER`, `DATE`) selon la nature des informations, garantissant la cohérence et prévenant les opérations invalides.
- **Standardisation** : Les numéros de téléphone sont normalisés, les emails validés, les montants uniformisés. Cette standardisation facilite les recherches et les analyses ultérieures.
- **Dédoublonnage** : Utilisation de `ROW_NUMBER() OVER (PARTITION BY ... ORDER BY ...)` pour identifier et éliminer les enregistrements dupliqués. En cas de doublon, seule la version la plus récente est conservée, garantissant la cohérence des soldes bancaires.
- **Contraintes d'intégrité** : Définition d'une clé primaire assurant l'unicité des enregistrements. Cette couche constitue un point de reprise fiable en cas d'échec des traitements ultérieurs.

4.1.3 Entrepôt de Données – Schéma en Étoile (Couche Gold)

La couche Gold implémente un schéma en étoile, paradigme fondamental du Data Warehousing. Ce modèle se caractérise par une table de faits centrale entourée de tables de dimension, offrant une structure intuitive et performante pour les analyses décisionnelles.

Tables de Dimension

Les dimensions encapsulent les contextes descriptifs nécessaires à l'analyse :

`DIM_CUSTOMER` Centralise les informations uniques sur les clients (nom, adresse, coordonnées, statut). En stockant ces données une seule fois, on évite leur répétition dans chaque transaction, réduisant l'espace de stockage et simplifiant les mises à jour.

`DIM_PRODUCT` Unifie les différents types de produits bancaires (comptes, prêts, cartes) sous une entité commune. Contient les caractéristiques produits (taux d'intérêt, frais, conditions d'éligibilité).

`DIM_DATE` Fournit une granularité temporelle complète (année, trimestre, mois, jour, indicateurs de jour ouvré/férié). Le pré-calcul de ces attributs évite des calculs coûteux lors des requêtes analytiques sur des millions de lignes.

Table de Faits

`FACT_TRANSACTIONS` constitue le cœur analytique du Data Warehouse :

- **Clés étrangères** : Références vers toutes les dimensions, établissant les relations du schéma en étoile.
- **Métriques natives** : Montants des transactions, soldes, montants de prêts, limites de crédit.
- **KPI calculés** : Indicateurs à valeur ajoutée absents des données sources (Risk Score, Profitability, Transaction Danger Score, Financial Runway).

- **Optimisation** : Structure dénormalisée pour maximiser les performances des requêtes analytiques et l'exploitation par les outils BI.

4.1.4 Table de Surveillance

`ALERTS_LOG` remplit une fonction critique de conformité et de sécurité. Elle capture automatiquement les événements suspects (fraudes potentielles, transactions anormales) avec leurs métadonnées (horodatage, type d'alerte, sévérité). Cette table répond aux exigences réglementaires en conservant une trace indélébile de toutes les anomalies détectées.

4.2 Packages PL/SQL

4.2.1 Package Technique : `PKG_DATA_CLEANING`

Ce package encapsule la logique de nettoyage dans des fonctions et procédures réutilisables:

`FN_STD_PHONE` Normalise les numéros de téléphone en supprimant les caractères non numériques et en conservant les 10 derniers chiffres. Cette standardisation facilite les recherches et les processus d'appel automatisés.

`FN_FIX_EMAIL` Valide la présence du symbole '@' dans les emails. Si absent, ajoute un domaine par défaut (`@unknown.com`) ou retourne `NULL`. Prévient les échecs des processus d'envoi d'emails.

`SP_CLEAN_STAGING` Moteur ETL principal entre Bronze et Silver. Utilise `ROW_NUMBER()` pour le dédoublonnage intelligent : si le fichier CSV contient deux versions d'une même transaction, seule la plus récente est conservée, garantissant la cohérence des soldes.

4.2.2 Package Métier : `PKG_BUSINESS_LOGIC`

Ce package crée de la valeur ajoutée en transformant les données brutes en intelligence actionnable.

Fonctions de Calcul des KPI

Ces fonctions traduisent des règles métier en formules opérationnelles :

1. `FN_GLOBAL_RISK_SCORE` : Calcule le ratio Dettes/Avoirs pour identifier les clients surendettés. Un score > 70 indique une situation préoccupante.

2. `FN_FINANCIAL_RUNWAY` : Estime combien de mois le client peut survivre avec son solde actuel. Indicateur prédictif pour anticiper les difficultés de paiement.

3. `FN_CUSTOMER_PROFITABILITY` : Quantifie la rentabilité client via le calcul des intérêts générés. Permet la segmentation pour cibler les efforts commerciaux.

4. `FN_TRANSACTION_DANGER_SCORE` : Détecte les transactions potentiellement frauduleuses basé sur le ratio transaction/solde et les patterns inhabituels.

Procédure de Peuplement : `SP_POPULATE_GOLD`

Cette procédure orchestre le transfert des données de Silver vers Gold en utilisant un curseur explicite.

Justification du traitement ligne par ligne :

Contrairement au traitement ensembliste privilégié en SQL, le curseur est ici nécessaire pour gérer la logique conditionnelle complexe des dimensions. Pour chaque transaction, la procédure doit :

1. Vérifier si le client existe dans `DIM_CUSTOMER`
2. Si non, le créer et récupérer son nouvel ID ; si oui, récupérer l'ID existant
3. Répéter cette logique pour `DIM_PRODUCT` et `DIM_DATE`
4. Calculer les KPI spécifiques à cette ligne
5. Insérer dans `FACT_TRANSACTIONS` avec toutes les clés étrangères

Cette approche garantit l'intégrité référentielle et permet de construire le schéma en étoile dynamiquement. Une approche ensembliste simple ne pourrait pas gérer ces opérations conditionnelles complexes.

Procédure : `SP_GEN_CALENDAR`

Génère l'ensemble de la dimension temporelle (2010-2030) via une boucle `WHILE`. Cette procédure est exécutée une seule fois lors de l'initialisation, créant environ 7 670 lignes qui garantissent qu'aucune date ne manque pour les jointures analytiques.

4.3 Mécanisme de Surveillance : Trigger

4.3.1 `TRG_DETECT_CRITICAL_RISK`

Ce trigger implémente un mécanisme de réaction automatique en temps réel. Il s'exécute automatiquement après chaque insertion dans `FACT_TRANSACTIONS`.

Logique de Détection

- `TRANSACTION_DANGER_SCORE` > 80 : Transaction individuelle anormale ou dangereuse
- `GLOBAL_RISK_SCORE` > 70 : Situation financière globale du client préoccupante

Lorsque les deux conditions sont satisfaites, le trigger insère immédiatement une alerte dans `ALERTS_LOG` avec un message descriptif, un horodatage et un niveau de sévérité.

Avantage Stratégique
Cette approche sépare l'analyse différée (effectuée par les analystes) de la sécurité immédiate (gérée automatiquement par la base). Même sans intervention humaine, le système enregistre les fraudes potentielles dès leur occurrence, garantissant qu'aucune situation critique ne passe inaperçue.

4.4 Flux de Données Global

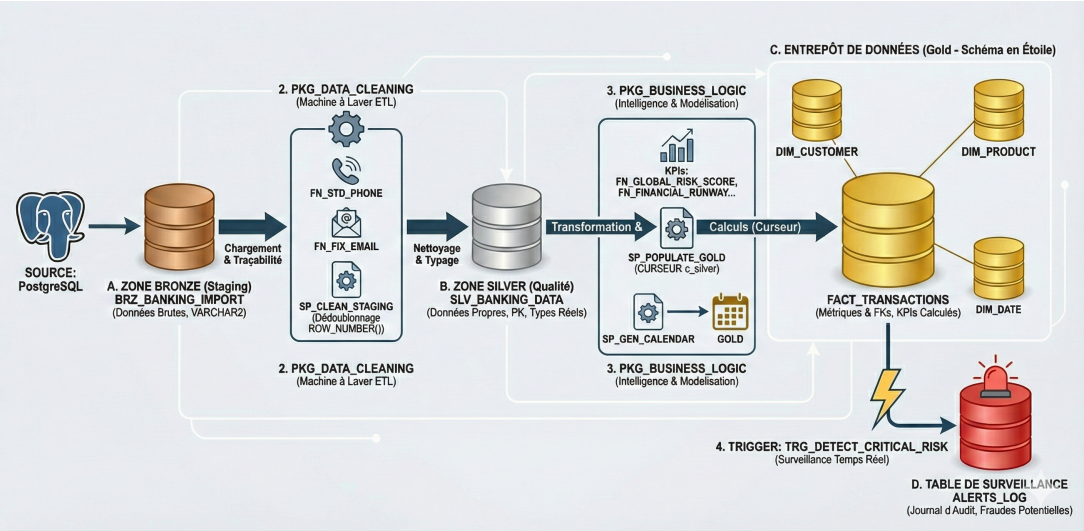


Figure 4.1: Pipeline de traitement des données : de PostgreSQL vers les alertes

Le processus complet suit cette chaîne de transformation :

$$\text{PostgreSQL} \xrightarrow{\text{Extraction}} \text{Bronze} \xrightarrow{\text{Nettoyage}} \text{Silver} \xrightarrow{\text{Intelligence}} \text{Gold} \xrightarrow{\text{Surveillance}} \text{Alertes}$$

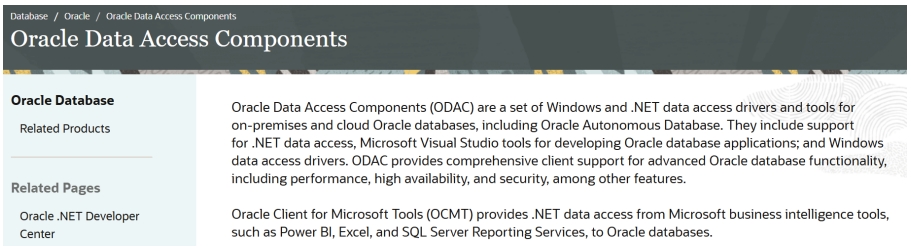
1. **Extraction** : Extraction des données de la base PostgreSQL source vers `BRZ_BANKING_IMPORT`
2. **Transformation** : Nettoyage et standardisation via `PKG_DATA_CLEANING` vers `SLV_BANKING_DATA`
3. **Enrichissement** : Calcul des KPI et distribution dans le schéma en étoile via `PKG_BUSINESS_LOGIC`
4. **Monitoring** : Détection automatique et journalisation des anomalies via `TRG_DETECT_CRITICAL_RISK`

Cette architecture garantit traçabilité, qualité des données et réactivité opérationnelle tout en optimisant les performances des analyses décisionnelles.

Chapter 5

Visualisation, Modélisation et Automatisation

5.1 Connexion Oracle – Power BI



Pour exploiter pleinement les données préparées dans notre Data Warehouse Oracle, il était nécessaire d'établir une connexion directe avec Power BI, outil de visualisation et d'analyse décisionnelle. Étant donné que notre base Oracle est installée localement (on-premises), la connexion ne pouvait pas se faire nativement.

Pour cela, nous avons utilisé Oracle Data Access Components (ODAC). Ce composant permet à Power BI de communiquer avec Oracle, d'extraire les vues de la couche Gold et de les rendre exploitables pour la création de rapports et d'analyses interactives. L'installation et la configuration de ODAC ont été indispensables: elles ont permis de résoudre les problèmes de compatibilité et de garantir que les données transformées soient accessibles en toute sécurité pour la visualisation.

5.2 Modélisation des données

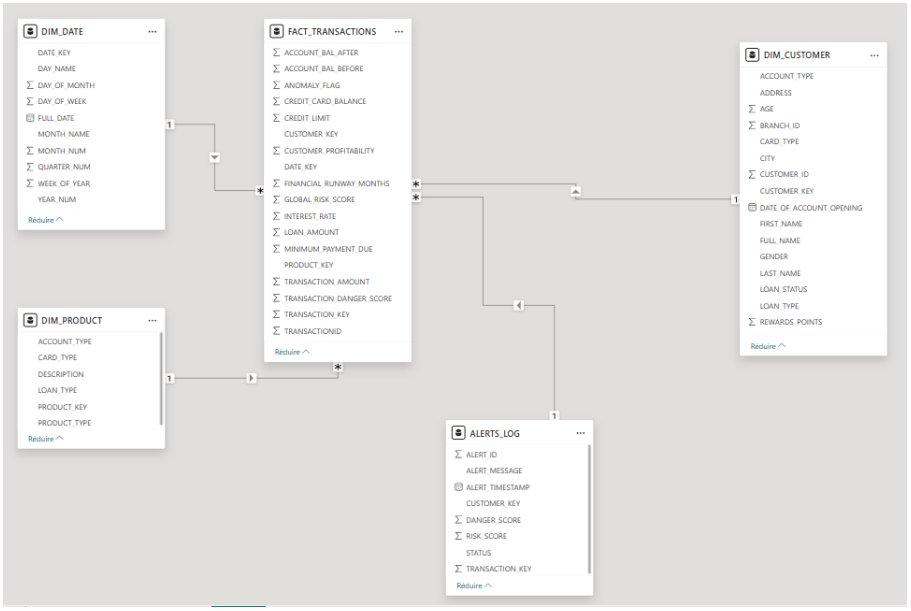


Figure 5.1: Le Data Model

La structure des données dans le Data Warehouse a été pensée pour optimiser l'analyse décisionnelle. Nous avons choisi un schéma en étoile, standard dans les systèmes décisionnels, afin de faciliter les calculs et les agrégations.

- **Table de faits principale :**

FACT_TRANSACTIONS : Elle centralise toutes les métriques importantes: montants des transactions, scores de risque et indicateurs de profitabilité.

- **Dimensions clés :**

DIM_DATE : analyse temporelle des transactions.

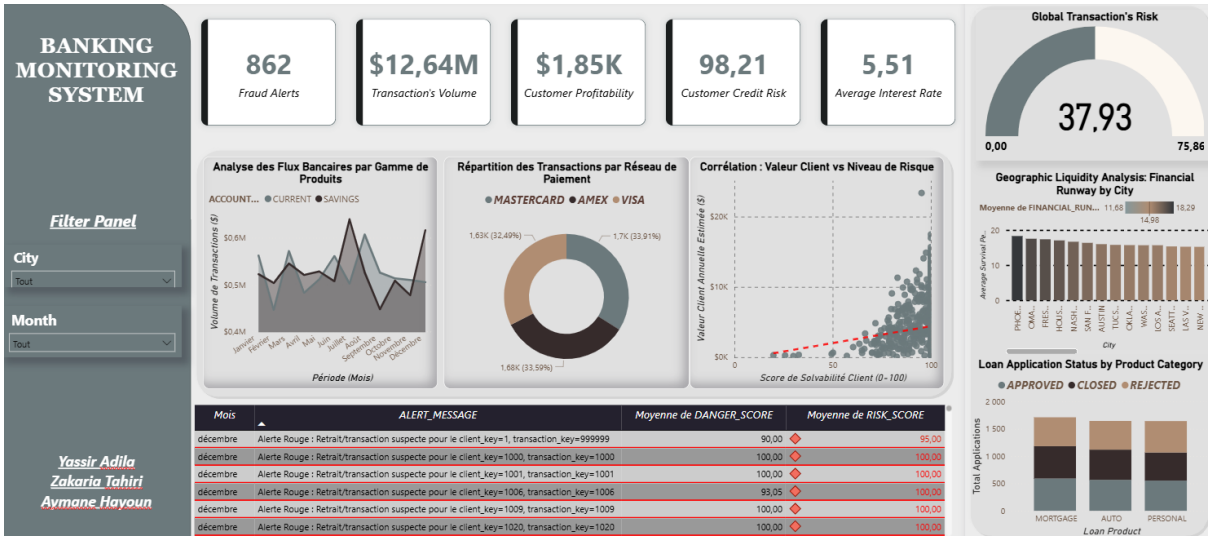
DIM_PRODUCT : segmentation des types de comptes et produits bancaires.

DIM_CUSTOMER : informations et profils clients.

ALERTS_LOG : journal des alertes générées automatiquement par les triggers PL/SQL pour détecter les comportements suspects.

Chaque ligne de la table ALERTS_LOG correspond à une alerte spécifique, ce qui permet un suivi précis des incidents et une priorisation des actions correctives.

5.3 Dashboard de monitoring bancaire



Le dashboard présenté constitue un outil décisionnel centralisé permettant une vision complète de l'activité bancaire et du niveau de risque associé aux transactions et aux clients. Il est structuré en plusieurs sections, chacune offrant des informations clés sur différents aspects de l'activité financière.

5.3.1 Indicateurs clés (KPI)

En haut du dashboard, cinq indicateurs principaux fournissent un aperçu immédiat:

- **Fraud Alerts (862)**: nombre total d'alertes de fraude détectées par le système.
- **Transaction's Volume ($12,64M)**: volume total des transactions traitées sur la période analysée.
- **Customer Profitability ($1,85K)** : profitabilité moyenne des clients, calculée à partir des marges sur les produits bancaires.
- **Customer Credit Risk (98,21)**: score moyen de risque de crédit, permettant d'identifier les clients présentant un potentiel de défaut élevé.
- **Average Interest Rate (5,51%)**: taux d'intérêt moyen appliqué aux différents produits financiers.

Un *gauge* supplémentaire montre le **Global Transaction's Risk (37,93)**, fournissant une mesure synthétique de l'exposition globale au risque des transactions.

5.3.2 Analyses visuelles

Le corps central du dashboard est organisé en trois types de visualisations principales:

Analyse des flux bancaires par gamme de produits Un graphique en ligne représentant le volume des transactions mensuelles pour chaque type de compte (Current et Sav-

ings). Il permet d'identifier les tendances saisonnières et les périodes de forte activité, facilitant la planification opérationnelle.

Répartition des transactions par réseau de paiement Un graphique circulaire (*pie chart*) affiche la proportion des transactions effectuées via MasterCard, AMEX et Visa. Cette visualisation aide à comprendre la répartition des moyens de paiement utilisés par les clients, utile pour la gestion des risques et la stratégie commerciale.

Corrélation: valeur client vs niveau de risque Un nuage de points montre la relation entre la valeur estimée annuelle d'un client et son score de solvabilité. La ligne de tendance permet de repérer les clients à forte valeur mais présentant un risque élevé, ce qui est crucial pour la priorisation des alertes et le suivi du portefeuille.

5.3.3 Analyses complémentaires

Analyse géographique (Financial Runway by City) Un histogramme montre la liquidité par ville, indiquant les zones où la stabilité financière est la plus élevée. Cette carte de chaleur aide la banque à orienter ses décisions sur le déploiement des ressources et la gestion du risque localisé.

Loan Application Status by Product Category Un graphique à barres illustre le statut des demandes de prêt par type de produit (Mortgage, Auto, Personal), avec les catégories Approved, Closed, Rejected. Cela permet d'analyser le succès des offres, les taux de rejet et de fermer le cycle décisionnel lié au crédit.

5.3.4 Tableau de monitoring opérationnel

La partie inférieure du dashboard présente les alertes critiques, avec pour chaque ligne:

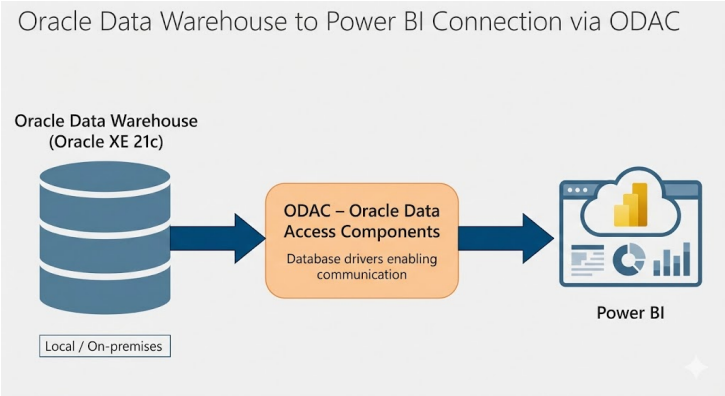
- Mois de détection
- Message détaillé de l'alerte
- Score de danger et score de risque

Les alertes rouges correspondent aux transactions suspectes identifiées automatiquement par le système PL/SQL. Cette section permet aux analystes de prioriser les interventions et d'agir rapidement, assurant la réactivité face aux incidents.

5.3.5 Interactivité et filtres

Le panneau de filtrage sur la gauche permet de sélectionner la ville et le mois, ce qui rend le dashboard interactif et adapté à des analyses ciblées. Cette fonctionnalité améliore la lisibilité et la pertinence des informations selon les besoins de l'utilisateur.

5.4 Automatisation et actualisation des rapports



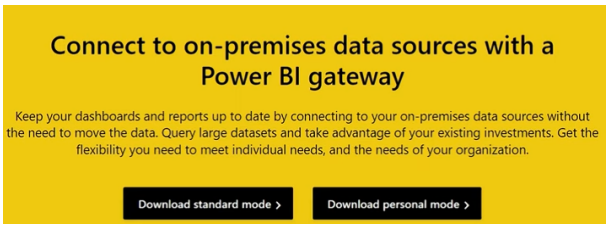
Pour garantir que notre tableau de bord bancaire reste toujours à jour et reflète en temps réel les dernières transactions et alertes, nous avons mis en place une **solution d'automatisation via Power BI On-Premises Data Gateway**. Cette architecture permet de connecter de manière sécurisée notre **base de données Oracle locale** au **service Power BI dans le cloud**, tout en conservant les données derrière notre pare-feu d'entreprise.

Le *On-Premises Data Gateway* agit comme un **pont sécurisé** entre le système local (notre base Oracle) et le service Power BI Service. Une fois configuré, il permet au service Power BI de :

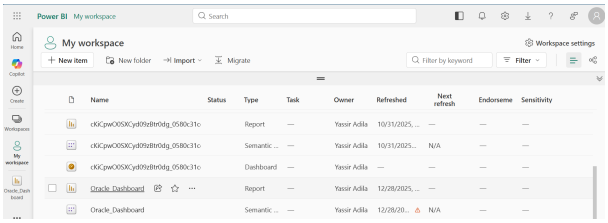
- Accéder à la base de données Oracle pour récupérer les nouvelles données.
- Actualiser automatiquement les jeux de données publiés sans intervention manuelle.
- Maintenir des rapports dynamiques et fiables accessibles aux utilisateurs autorisés.

Concrètement, l'automatisation fonctionne comme suit :

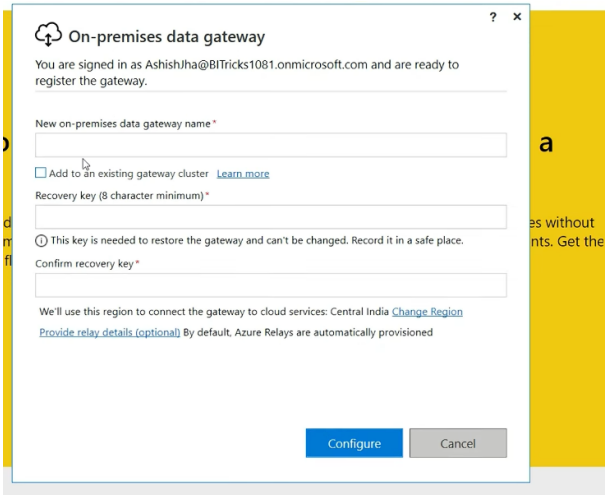
1. **Installation du Gateway :** le composant *On-Premises Data Gateway* est installé sur un serveur Windows toujours disponible, avec le client Oracle requis pour assurer la connectivité entre Power BI et la base Oracle.

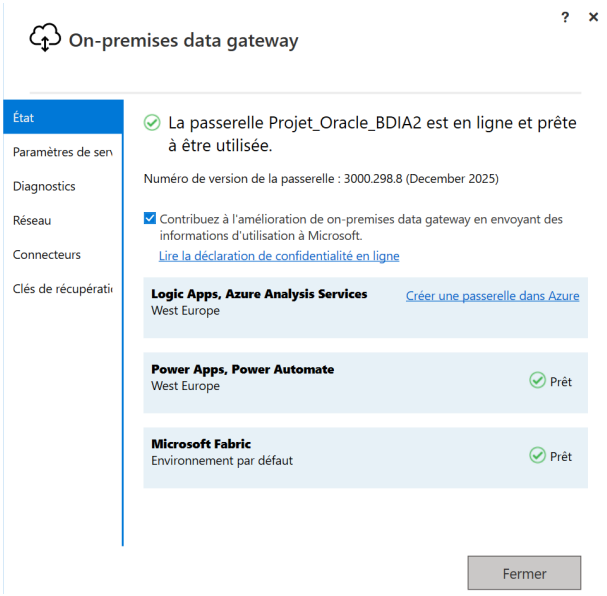


2. **Publication du rapport Power BI :** le rapport est développé avec Power BI Desktop en se connectant à la base Oracle. Une fois finalisé, il est publié dans le service Power BI associé à notre compte professionnel.

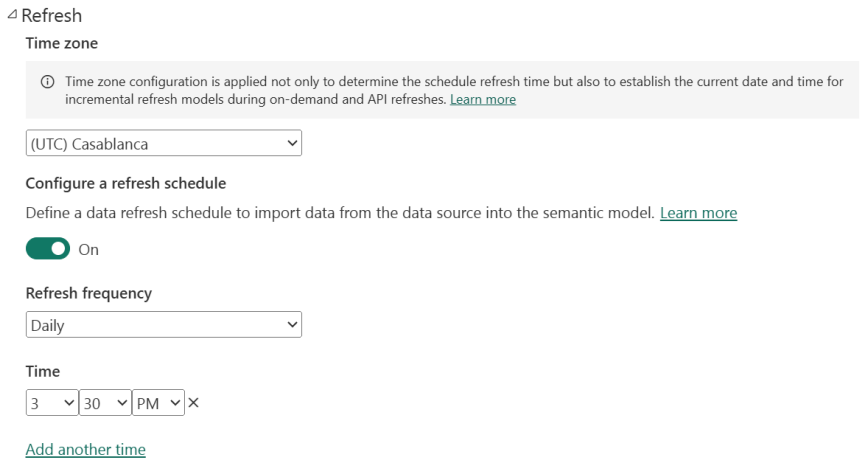


3. **Configuration de la connexion dans Power BI Service :** dans les paramètres du rapport publié, nous activons l'utilisation du *On-Premises Data Gateway* et définissons les informations d'accès à la source Oracle (serveur, base, authentification).





4. **Planification de l'actualisation :** une *actualisation planifiée* (*scheduled refresh*) est configurée pour récupérer automatiquement les nouvelles données à partir de la base locale via le gateway, garantissant que le tableau de bord affiche toujours les données les plus récentes.



Grâce à cette solution :

- Les rapports sont automatiquement actualisés à intervalles réguliers dès qu'une nouvelle transaction ou alerte est enregistrée dans la base Oracle.
- Les utilisateurs disposent d'un tableau de bord dynamique et fidèle à l'état réel des comptes, sans nécessiter d'intervention manuelle.
- L'approche assure la continuité et la fiabilité du suivi bancaire, tout en démontrant la possibilité d'intégrer des bases locales dans des systèmes décisionnels modernes basés sur le cloud.

Cette intégration entre une **infrastructure locale sécurisée** et un **service analytique moderne** illustre une pratique efficace de *Business Intelligence* dans les environnements professionnels, en tirant parti des capacités de Power BI pour automatiser la diffusion et l'actualisation des insights métiers.

Chapter 6

Conclusion

Le projet de mise en place d'un *Data Warehouse* bancaire avec Oracle et Power BI a permis de démontrer l'importance d'une architecture décisionnelle efficace pour le suivi et l'analyse des transactions financières.

Grâce à l'intégration des données locales via l'On-Premises Data Gateway et l'automatisation des rapports, nous avons pu :

- Assurer la disponibilité et la fiabilité des informations en temps réel.
- Offrir aux utilisateurs des tableaux de bord dynamiques pour faciliter la prise de décision.
- Démontrer la possibilité d'intégrer des systèmes traditionnels et des solutions cloud modernes dans un environnement sécurisé.

Ce projet illustre également les bonnes pratiques de *Business Intelligence* et ouvre la voie à des améliorations futures, notamment la mise en place de systèmes avancés de détection de fraude, l'optimisation des performances des requêtes OLAP et l'enrichissement des indicateurs financiers pour une analyse prédictive plus fine.

Chapter 7

Références

Bibliography

[1] Microsoft. *Power BI On-Premises Data Gateway: Manage Oracle data sources.* [Online]. Available: `https://learn.microsoft.com/en-us/power-bi/connect-data/service-gateway-onprem-manage-oracle`

[2] Microsoft. *Scheduled refresh in Power BI.* [Online]. Available: `https://learn.microsoft.com/en-us/power-bi/connect-data/refresh-scheduled-refresh`

[3] Oracle Corporation. *Oracle Data Warehouse Guide.* [Online]. Available: `https://docs.oracle.com/en/database/oracle/oracle-database/`

[4] Kimball, R., & Ross, M. *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling.* 3rd Edition, Wiley, 2013.

[5] YouTube. *How to Use Power BI On-Premises Data Gateway with Oracle.* [Online]. Available: `https://www.youtube.com/watch?v=i4OALAQ7qz4`